Windows Telemetry and Splunk Detection Lab

Final report for Windows telemetry and Kali scan detection

Soranut Srisilapaudom
Evidence collected 16 to 20 September 2026
Version 2.0 | Issued 20 September 2026

Outcome

I built a Windows telemetry pipeline into Splunk, created and tested three scheduled detections, investigated the resulting events, and built a four-panel dashboard. The evidence shows successful collection of Windows Security, Sysmon, and PowerShell logs from WIN11-01. The repeated failed-logon, encoded PowerShell, and possible TCP port-scan alerts all fired automatically.

This report documents the completed Windows telemetry and Kali TCP scan use cases. Kali scanning produced Windows Filtering Platform audit events, which reached Splunk and triggered the third detection. Domain controller log collection remains outside the verified scope.

Verified milestone	Evidence
Windows telemetry ingestion	Security, Sysmon and PowerShell events searchable in soclab
Failed-logon detection	Replacement per-account rule fired at 18:15:01 UTC on 18 September
Encoded PowerShell detection	Scheduled rule fired at 19:40:01 UTC on 18 September
Investigation	Process IDs and timestamps connect Sysmon creation to script-block content
TCP scan detection	Scheduled alert fired at 16:10:00 UTC on 20 September
Dashboard	Four panels including 405 blocked events across 98 destination ports

How to read the results

All tests described here were authorized activity in my own virtual lab. Failed logons and encoded PowerShell execution are observable behaviors; neither alone proves malicious activity. Event times in the investigation timeline are UTC. The dashboard totals represent a selected historical window, not the number of alerts.

Evidence basis

Results are based on the supplied Splunk and Windows screenshots and the configuration work recorded during the build. This is a lab report, not an independent forensic acquisition. Original event exports and screenshot hashes were not collected as part of this phase.

Lab architecture and collection scope

The lab uses VirtualBox with a NAT adapter for internet access and a host-only adapter for communication between lab systems. Splunk runs on Ubuntu. WIN11-01 collects its local event channels through the Splunk Universal Forwarder and sends data to the receiver on TCP 9997.

System	Role	Lab address	State
DC01	Windows Server 2025 AD DS and DNS	192.168.56.10	Domain configured
WIN11-01	Windows 11 Pro monitored endpoint	192.168.56.20	Telemetry verified
SPLUNK01	Ubuntu 24.04 LTS with Splunk Enterprise	192.168.56.41	Ingestion verified
Kali VM	Nmap test source	192.168.56.30	Scan verified

The Active Directory domain is soclab.local, with NetBIOS name SOCLAB. The endpoint is domain joined. The test process evidence identifies the local account WIN11-01\jame. DC01 provides domain services but its own logs have not yet been shown arriving in Splunk.

Data flow

Windows event channels on WIN11-01 are read by SplunkForwarder, sent to 192.168.56.41:9997, and indexed in soclab. Splunk add-ons provide field extraction for searches, alerts, and dashboard panels. The web interface is available at http://192.168.56.41:8000.

Event channel	Events used	Purpose
Security	4625; 5152	Investigate failed logons and blocked TCP packets
Microsoft-Windows-Sysmon/Operational	1	Record process creation and command lines
Microsoft-Windows-PowerShell/Operational	4104	Inspect script-block content

Installed components

Splunk Enterprise and the Windows Universal Forwarder were version 10.4.3. The Splunk server had Splunk Add-on for Microsoft Windows 11.0.2 and Splunk Add-on for Sysmon 5.0.1 installed and enabled. Sysmon ran as the Sysmon64 service on WIN11-01.

Scope boundary

The implemented pipeline uses the Universal Forwarder directly. Windows Event Forwarding was discussed as a possible approach but was not implemented in this phase. Kali scan visibility was validated using Security Event 5152. Sysmon network Event 3 and ingestion of a separate firewall text log were not used or validated.

Collection configuration and access

The endpoint input configuration was stored in C:\Program Files\SplunkUniversalForwarder\etc\system\local\inputs.conf. The following configuration represents the three enabled channels used in the lab.

```
[WinEventLog://Security]
disabled = 0
index = soclab
renderXml = true
start_from = oldest
current_only = 0

[WinEventLog://Microsoft-Windows-PowerShell/Operational]
disabled = 0
index = soclab
renderXml = true
start_from = oldest
current_only = 0

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
disabled = 0
index = soclab
renderXml = true
start_from = oldest
current_only = 0
```

disabled = 0 enables an input. index = soclab selects the destination index. renderXml = true requests XML event rendering. start_from = oldest specifies the initial reading direction; current_only = 0 allows collection of retained events rather than only events generated while the input runs. Existing checkpoints affect where collection resumes. [1]

Resolved Sysmon access problem

Sysmon events existed locally, but the forwarder initially encountered subscription error 5. The forwarder ran as NT SERVICE\SplunkForwarder. Adding that identity to Event Log Readers and restarting the service resolved the collection issue in this lab.

```
net localgroup "Event Log Readers" "NT SERVICE\SplunkForwarder" /add
Restart-Service SplunkForwarder
Get-Service SplunkForwarder
```

A harmless cmd.exe test containing SOC-LAB-SYSMON-PERMISSION-TEST then appeared in Splunk. Parsed fields included Image, CommandLine, User, and ParentImage. Later process evidence independently confirmed continued collection.

Parsing observations

Screenshots contain both WinEventLog and XmlWinEventLog source prefixes following the configuration change. Different prefixes alone do not establish duplicate ingestion. Searches use channel-aware wildcards such as source="*Sysmon*". Duplicate-event testing was not completed.

Detection one repeated failed logons

The final rule detects at least five failed-logon events for the same host, target domain, and target username during the preceding 15 minutes. It generalizes the earlier rule that matched only SOC_Test_NoUser.

```
index=soclab host="WIN11-01" EventCode=4625 earliest=-15m latest=now
| fillnull value="unknown" TargetDomainName
| stats count AS failed_attempts
    values(IpAddress) AS source_ips
    values(LogonType) AS logon_types
    min(_time) AS first_seen
    max(_time) AS last_seen
    by host TargetDomainName TargetUserName
| where failed_attempts >= 5
| convert ctime(first_seen) ctime(last_seen)
```

stats creates a separate summary row for each account grouping. values preserves distinct source addresses and logon types; min and max record the first and last observed event. where removes groups below the threshold. convert makes the timestamps readable. An alert condition of results greater than zero therefore means at least one account met the threshold. [2]

Setting	Final configuration
Title	LAB - Repeated Failed Logons - Per Account
Schedule	Scheduled; `*/5 * * * *`; every five minutes
Search window	Last 15 minutes
Trigger	Number of results greater than 0; for each result
Throttle	15 minutes on host,TargetDomainName,TargetUserName
Action and severity	Add to Triggered Alerts; Medium
Permissions and expiry	Private; 24-hour triggered-alert retention setting

Test method

I repeatedly attempted to launch cmd.exe using deliberately nonexistent local usernames and entered a fake password. The expected RUNAS error 1326 appeared. The failure itself was the test stimulus; creating the nonexistent accounts was unnecessary.

```
runas /user:WIN11-01\SOC_Test_SecondUser cmd.exe
```

Windows Security Event 4625 supplied the target account and failure context. In the inspected events, ::1 was the local IPv6 loopback address and LogonType 2 represented interactive logon. These local tests do not demonstrate remote RDP brute force. [3]

Failed logon evidence and findings

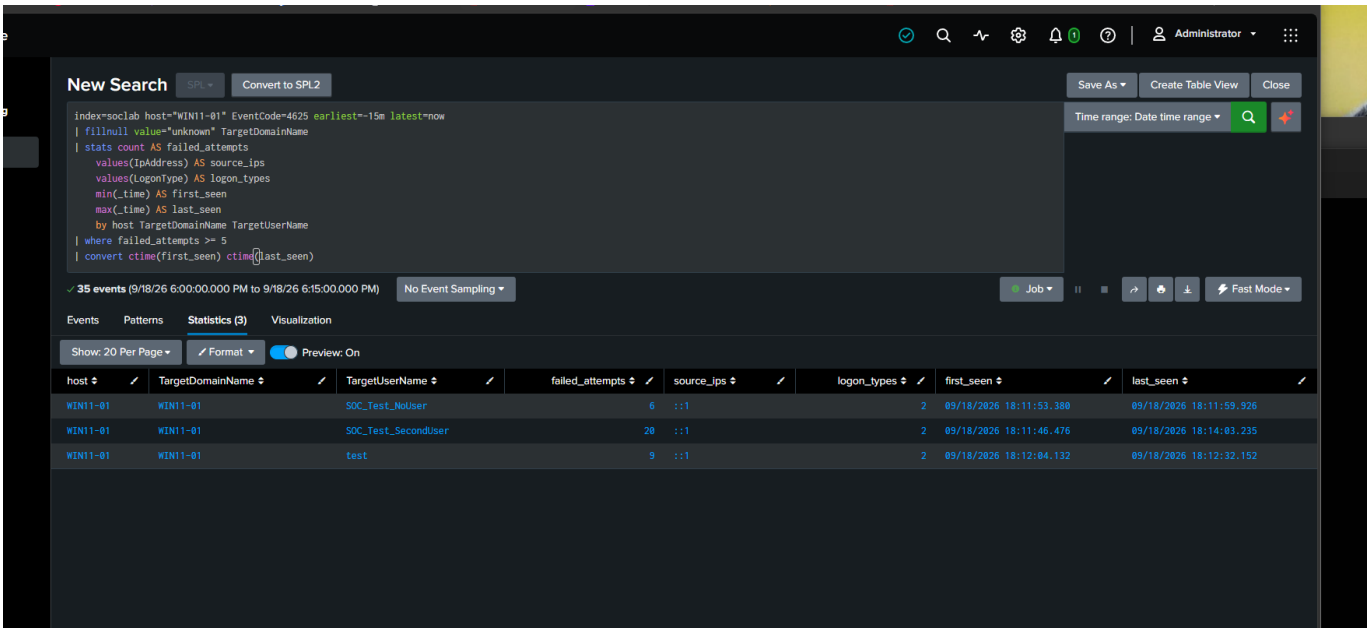


Figure 1. The 18:00 to 18:15 UTC search window returned 35 events and three qualifying account rows.

Target account	Failed attempts	Observed source and type
SOC_Test_NoUser	6	::1; LogonType 2
SOC_Test_SecondUser	20	::1; LogonType 2
test	9	::1; LogonType 2

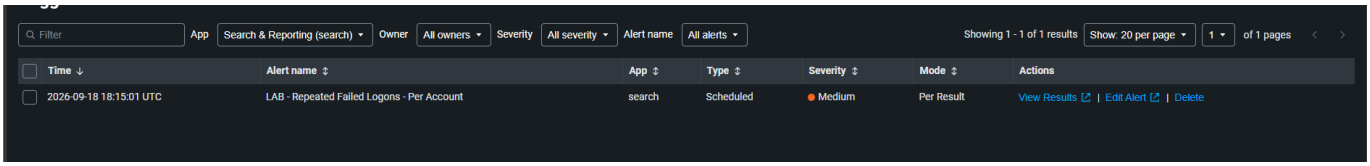


Figure 2. The replacement rule appears in Triggered Alerts at 18:15:01 UTC, with Medium severity and Per Result mode.

Interpretation

The query separated the three target accounts and retained each because its count met the five-event threshold. The scheduled trigger demonstrates that the saved alert evaluated matching data and recorded a triggered-alert entry. A listing row should not be treated as a count of every underlying event or account.

The 35 events above belong to one 15-minute validation window. The later dashboard total of 98 covers a longer period and includes additional tests. These values measure different windows and are not contradictory.

Rule cleanup

During development, four similar saved alerts existed. They were deleted at my request and replaced with one final per-account rule. This reduced duplicate rule execution. The displayed throttle configuration is recorded, but a separate controlled throttle-suppression test was not performed.

Detection two encoded PowerShell

The second rule detects powershell.exe process creation with the full -EncodedCommand switch. The test command only prints a unique marker. Base64 is an encoding; its presence alone is not proof of malicious execution.

Harmless test command

```
$labCommand = "Write-Output 'SOC-LAB-ENCODED-TEST'"
$labBytes = [System.Text.Encoding]::Unicode.GetBytes($labCommand)
$labEncoded = [Convert]::ToBase64String($labBytes)
powershell.exe -NoProfile -EncodedCommand $labEncoded
```

The script converts the command to UTF-16LE bytes, Base64-encodes those bytes, and starts a child PowerShell process to execute the command. -NoProfile skips PowerShell profile loading. Expected console output is SOC-LAB-ENCODED-TEST. [4]

Detection search

```
index=soclab host="WIN11-01" source="*Sysmon*" EventCode=1
    earliest=-15m latest=now
Image="*powershell.exe" CommandLine="*-EncodedCommand*"
| table _time host User Image CommandLine ParentImage ProcessId
| sort - _time
```

Sysmon Event 1 records process creation, including command-line and parent-process context. This search intentionally matches the full switch used in the lab. Short forms such as -enc and other PowerShell hosts are outside this rule's tested coverage. [5]

Setting	Recorded configuration
Title	LAB - PowerShell Encoded Command
Schedule and window	Scheduled; `*/5 * * * *`; Last 15 minutes
Trigger	Number of results greater than 0; for each result
Action and severity	Add to Triggered Alerts; Medium
Planned throttle configuration	15 minutes on host,CommandLine
Validation	Automatic firing verified; throttle behavior not separately tested

Using host and CommandLine as suppression fields can suppress a repeated identical test command on the same endpoint. The successful trigger establishes scheduling and detection; it does not independently prove every saved throttle setting.

PowerShell investigation timeline

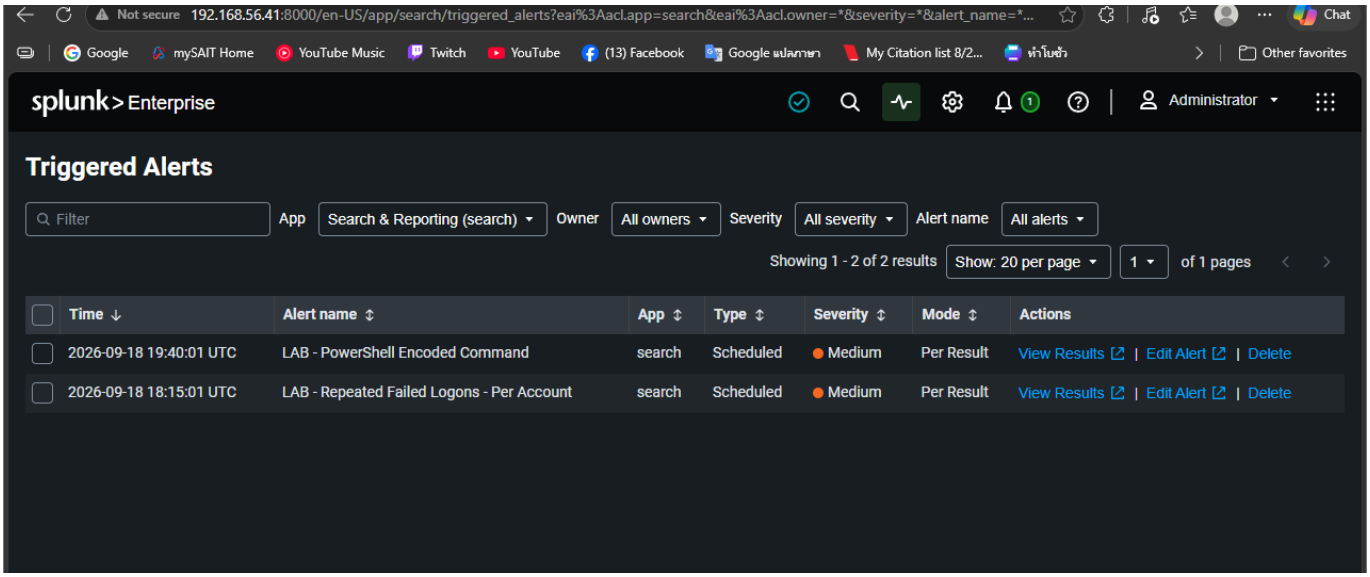


Figure 3. Triggered Alerts records the encoded PowerShell rule at 19:40:01 UTC on 18 September 2026.

UTC time on 18 September	Evidence	Meaning
19:37:24.332	PowerShell 4104; PID 8668	Parent session assigns the test command
19:37:26.348	Sysmon 1; PID 6860	Child powershell.exe starts with -EncodedCommand
19:37:26.674	PowerShell 4104; PID 6860	Child logs the decoded Write-Output command
19:40:01	Splunk triggered alert	Scheduled detection is recorded

Correlation and conclusion

The Sysmon and PowerShell events share WIN11-01 and process ID 6860, and are separated by approximately 0.326 seconds. Together they connect the encoded process launch to the readable script content. Matching a PID alone over a long period would be insufficient because process IDs can be reused.

The process ran as WIN11-01\jame and its parent image was powershell.exe, consistent with launching the child from the existing PowerShell session. The decoded command was Write-Output 'SOC-LAB-ENCODED-TEST'. The evidence is consistent with the authorized benign test.

The alert fired about 2 minutes 35 seconds after process creation, consistent with its next five-minute scheduled run. That delay includes scheduling and is not a measured ingestion-latency benchmark.

Investigation search

```
index=soclab host="WIN11-01" source="*PowerShell*" EventCode=4104
    earliest=-24h latest=now "SOC-LAB-ENCODED-TEST"
| table _time host ProcessID ScriptBlockText ScriptBlockId
| sort - _time
```

The 24-hour range was used during the investigation on 18 September. To retrieve this evidence later, use an absolute range covering that date or a suitable wider range. PowerShell Script Block Logging records processed script content in Event 4104. [6]

Script block evidence

Four matching events were visible: preparation and decoded content for each of two tests. The latest child had PID 6860; the earlier child had PID 2896. Four script-block records therefore do not mean four encoded process launches.

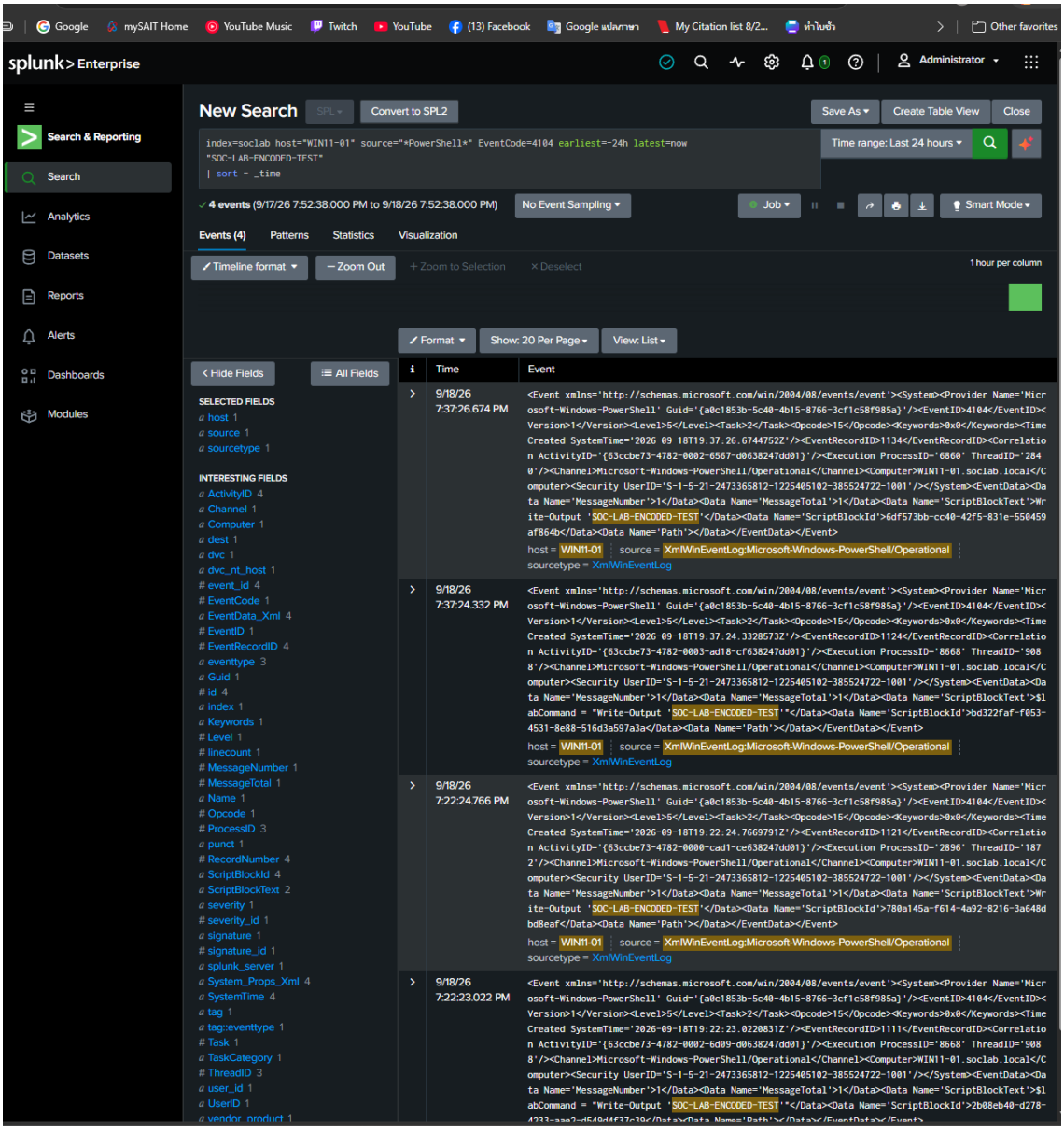


Figure 4. Original investigation screenshot showing the parent assignment and child script text for both tests.

The raw XML exposes ScriptBlockText and Execution ProcessID. The latest child event at 19:37:26.674 contains the readable Write-Output command. This supports the benign test conclusion while preserving the fact that the detection correctly matched encoded execution.

Dashboard and historical results

The saved dashboard is titled SOC Lab - Windows Security Overview. All three panels were shown with a global Last 7 days selection. The screenshot below documents the completed view on 19 September 2026.



Figure 5. Saved dashboard with failed-logon totals, hourly timeline and two encoded PowerShell process records.

Panel	Observed result
Failed Logons by Account	SOC_Test_NoUser 52; SOC_Test_SecondUser 31; test 9; TEST 6
Failed Logons Over Time	Hourly counts show spikes during the test sessions
PowerShell Encoded Command Activity	Two process records; PIDs 6860 and 2896

The account panel totals 98 failed-logon events. It counts all matching failures in the selected period; it does not apply the alert's five-in-15-minutes threshold. The text values test and TEST appear as separate groups in the current search. No account-name normalization was added.

Resolved dashboard time mismatch

Initially the dashboard was empty even when its global picker showed Last 7 days. The panel data source still used Static / Last 15 minutes. Connecting the data source to the global time input restored the results. The global picker controls searches only when their time settings use the appropriate input tokens. [7]

The dashboard supports historical review. It does not replace investigation of the raw events, and an empty recent window is not proof that collection has failed. The matching alert windows remained at 15 minutes.

Dashboard searches and reusable logic

The dashboard searches omit inline earliest and latest restrictions so the configured global time range can control them. The searches below reproduce the implemented panel logic.

Failed logons by account

```
index=soclab host="WIN11-01" source="*Security" EventCode=4625
| stats count AS failed_attempts by TargetUserName
| sort - failed_attempts
```

This aggregation answers which target account names received the most failures. The bar chart orders them from the highest count to the lowest.

Failed logons over time

```
index=soclab host="WIN11-01" source="*Security" EventCode=4625
| timechart span=1h count AS failed_attempts
```

timechart groups the matching events into one-hour intervals. Each point is a count for that hour, not a rolling 15-minute detection threshold. [8]

Encoded PowerShell activity

```
index=soclab host="WIN11-01" source="*Sysmon*" EventCode=1
Image="*powershell.exe" CommandLine="*-EncodedCommand*"
| sort - _time
| table _time host User CommandLine ParentImage ProcessId
```

The table retains the command and parent-process context for review. It lists matching process events, not PowerShell script-block records. Field capitalization follows the extracted fields observed in this lab: ProcessId for Sysmon and ProcessID for the PowerShell investigation.

Alert schedule versus dashboard range

The cron expression `*/5 * * * *` schedules a run every five minutes. The 15-minute alert window determines which event timestamps are searched by that run. Throttling controls repeated alert actions, while the dashboard global picker controls the historical data displayed. These are separate settings. [9]

Troubleshooting and validation limits

Problems resolved

Sysmon subscription error 5 was resolved by granting Event Log Readers membership to the forwarder service identity. Installing the Windows and Sysmon add-ons made the relevant event fields available for structured searches. Several similar saved failed-logon alerts were consolidated into one replacement rule.

A PowerShell alert initially displayed Next scheduled time as None. A later automatic trigger at 19:40:01 UTC confirmed that the rule executed; the exact reason for the earlier display was not established. Old test events falling outside a 15-minute range also explained some empty searches.

The dashboard's empty result was traced directly to its static 15-minute data-source setting. Changing it to the global time input restored the historical results. An unrelated allowedDomainList email warning did not prevent the Add to Triggered Alerts action; email notifications were not configured for these tests.

Limits of the completed validation

All three positive test cases and automatic triggers were verified. A controlled negative test, ingestion deduplication test, and independent test of throttle suppression were not completed. Systemd boot persistence for Splunk was discussed but was not evidenced as verified in this report.

The failed-logon test was local and used nonexistent usernames. It does not demonstrate a remote RDP attack, successful compromise, password spraying, or lateral movement. The encoded PowerShell rule detects the full switch only, is scoped to WIN11-01, and can match legitimate administrative activity.

The results describe a small lab and do not establish production detection coverage, measured false-positive rates, or performance under load. Saved alert results also have limited retention, so the report preserves the observed screenshots and transcribed findings.

Completed Kali extension

The Kali extension completed on 20 September. Two Nmap TCP connect scans targeted WIN11-01. Event 5152 supplied source address, destination address and destination port evidence, and the scheduled scan alert fired. The following sections preserve the commands, timing and observed results.

The scan rule counts distinct blocked TCP destination ports for each source and destination pair. Its 20-port threshold is a lab starting point, not a measured production baseline. It does not cover UDP, scans producing no blocked-packet records, or low-volume scans below the threshold. DC01 log forwarding remains a separate outstanding task.

Final assessment

The lab achieved its intended workflow: collect endpoint telemetry, detect three defined behaviors, record scheduled alerts, investigate their context, and present historical activity in a dashboard. The available evidence verifies the positive detection path; negative-control and independent throttle tests remain useful follow-up work.

Kali networking and audit preparation

Kali used eth0 at 10.0.2.15/24 for NAT and eth1 at 192.168.56.30/24 for the host-only lab network. NetworkManager identified the eth1 connection as SOC-LAB. The terminal hostname shown was got; KALI01 was only a planning label, not a verified hostname.

```
ip -br address
nmcli connection show
ping -c 4 192.168.56.20
ping -c 4 192.168.56.41
```

Splunk at .41 initially answered ping while WIN11-01 at .20 did not. Creating the scoped ICMP rule first failed with Access is denied and Windows System Error 5. Running PowerShell as Administrator resolved the permission issue. A subsequent Kali test received three Windows replies with 0% loss before it was stopped.

```
New-NetFirewallRule -DisplayName "LAB - Allow ping from Kali"
 -Direction Inbound -Protocol ICMPv4 -IcmpType 8
 -RemoteAddress 192.168.56.30 -Action Allow -Profile Any
```

The command above is displayed on multiple lines for readability; enter it as one line. It allows inbound IPv4 echo requests from Kali only. Windows Firewall remained enabled. [10]

Windows network audit configuration

```
auditpol /set /subcategory:"Filtering Platform Packet Drop" /failure:enable
auditpol /set /subcategory:"Filtering Platform Connection" /failure:enable
auditpol /get /subcategory:"Filtering Platform Packet Drop"
auditpol /get /subcategory:"Filtering Platform Connection"
```

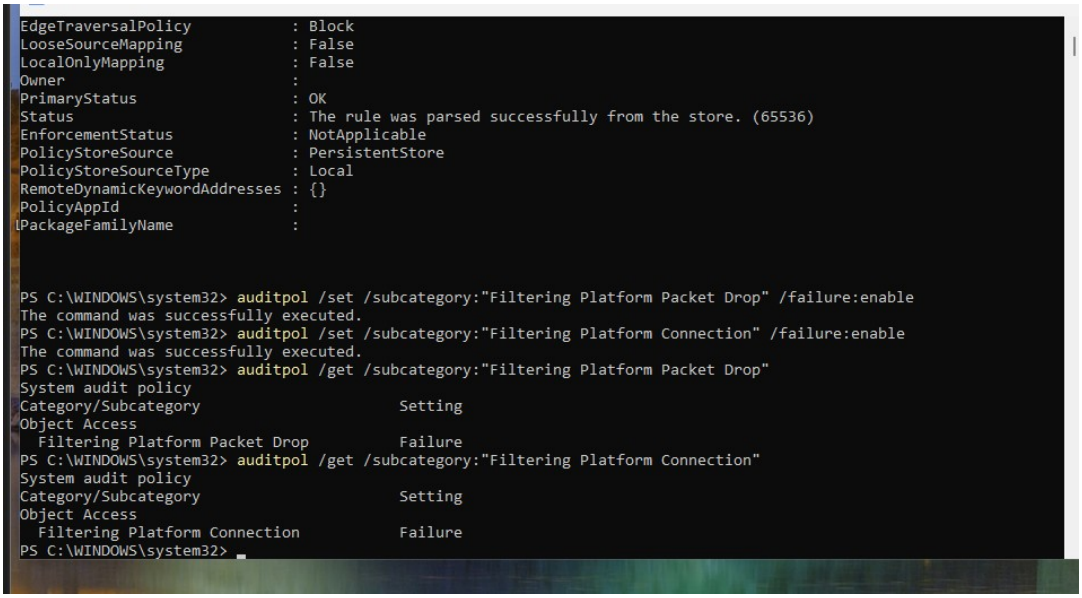


Figure 6. Both audit subcategories show Failure after successful configuration.

Event 5152 records blocked packets; Event 5157 records blocked connections. Both failure audit categories were enabled, but the scan detection evidence in this report is Event 5152. The existing Security log input transported these events to Splunk. [11, 12]

Controlled Nmap scan results

I ran Nmap 7.95 from Kali against WIN11-01 only. The TCP connect scan checked the 100 most common TCP ports. It did not perform vulnerability scripts, password attempts or exploitation. The displayed service names are port-based labels; no service-version scan was requested. [13]

```
date -u
nmap -sT -Pn -n --top-ports 100 --reason
 -oN kali-win11-scan.txt 192.168.56.20
```

Enter the Nmap command as one line. -sT selects TCP connect scanning; -Pn skips host discovery; -n skips DNS resolution; --reason explains port states; -oN writes normal text output. The repeat test used the filename kali-win11-alert-test.txt.



Figure 7. Two completed scans show 98 filtered ports and two open ports.

Run on 20 September	Duration	Result
09:55 MDT / 15:55 UTC	3.86 seconds	135/tcp and 3389/tcp open; 98 filtered
10:10 MDT / 16:10 UTC	2.03 seconds	135/tcp and 3389/tcp open; 98 filtered

The initial pasted date command produced a zsh bad-pattern error, but the separately entered date command and both Nmap commands succeeded. Filtered means Nmap could not determine whether those ports were open; the Windows audit evidence supports packet blocking for this test.

Detection three possible TCP port scan

The rule detects at least 20 distinct blocked TCP destination ports within the preceding 15 minutes for each host, source IP and destination IP combination. WIN11-01 is the target; the source is not restricted to Kali. Counting distinct ports limits the effect of retransmissions on the detection threshold.

```
index=soclab host="WIN11-01" source="*Security"
EventCode=5152 earliest=-15m latest=now
| rex field=_raw "<Data Name=['\"]SourceAddress['\"]>(?<scan_src_ip>[^<]+)"
| rex field=_raw "<Data Name=['\"]DestAddress['\"]>(?<scan_dest_ip>[^<]+)"
| rex field=_raw "<Data Name=['\"]DestPort['\"]>(?<scan_dest_port>[^<]+)"
| rex field=_raw "<Data Name=['\"]Protocol['\"]>(?<scan_protocol>[^<]+)"
| search scan_dest_ip="192.168.56.20" scan_protocol="6"
| stats count AS blocked_events
    dc(scan_dest_port) AS unique_dest_ports
    values(scan_dest_port) AS destination_ports
    min(_time) AS first_seen max(_time) AS last_seen
    by host scan_src_ip scan_dest_ip
| where unique_dest_ports >= 20
| convert ctime(first_seen) ctime(last_seen)
```

rex extracts values from the XML event data into explicit scan fields. Protocol 6 selects TCP. stats groups the records; count measures events and dc measures distinct ports. values retains the observed port list. min and max preserve the first and last event times; convert formats them for display. Line breaks are optional because pipes delimit the commands. [2, 14]

Setting	Value and verification
Saved title	Possible TCP Port Scan - WIN11-01; visible in Triggered Alerts
Search threshold	At least 20 distinct destination ports; visible in saved result SPL
Schedule	Scheduled verified; five-minute cron */5 * * * * instructed
Lookback	15 minutes; saved job window 15:55 to 16:10 UTC
Trigger	Number of results greater than 0 instructed; Per Result verified
Action and severity	Triggered Alerts entry verified; Medium verified
Throttle	15 minutes on host,scan_src_ip,scan_dest_ip instructed
Expiry	24 hours instructed

A final settings-dialog screenshot was not supplied for this rule. The job and trigger prove automatic execution and the saved search behavior; they do not independently verify the configured cron, retention or throttle fields. Throttle suppression was not separately tested.

Scan investigation and scheduled alert evidence

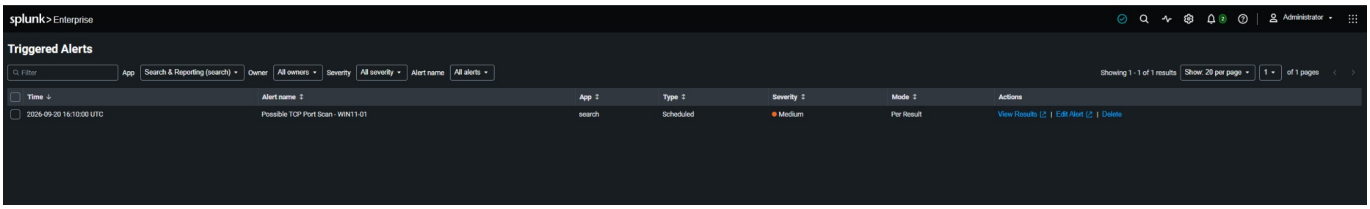


Figure 8. The scheduled scan alert fired at 16:10:00 UTC on 20 September 2026.

Evidence from the saved alert result	Observed value
Host and traffic direction	WIN11-01; Kali 192.168.56.30 to 192.168.56.20
Event type	Security Event 5152; TCP protocol 6
Blocked events / distinct ports	206 / 98
First event UTC	20 September 2026 15:55:33.682
Last event UTC	20 September 2026 15:55:37.019
Observed event span	3.337 seconds
Scheduled search window	15:55:00 to 16:10:00 UTC

Which scan triggered the alert

The saved job contains first and last event times from 15:55, so the 16:10 alert detected the initial scan. The second Nmap command began around 16:10 and is not represented in that saved result. An alert timestamp must not be substituted for the timestamp of the activity it detected.

Comparison with Nmap

The initial Nmap result reported 98 filtered ports and two open ports. The Windows summary contains 98 distinct blocked destination ports, which is consistent with that result. Equal totals alone do not prove a full port-by-port match; the complete scan text and event exports were not supplied for an exact comparison.

The 206 audit events do not mean 206 scanned ports. A scan can create multiple packets or audit records for a destination port. The two open ports were 135/tcp and 3389/tcp, labelled msrpc and ms-wbt-server by Nmap. Open RDP reachability is not evidence of a successful login or compromise.

Evidence interpretation

The repeated connection attempts to many ports, the Kali source address, the short event span and the recorded Nmap command support the conclusion that this was the authorized lab scan. A port-scan alert in another environment would still require investigation of the source and its purpose.

Supporting screenshots: 3763b311-e78d-48c4-ad3c-21a1fa05f38c.png shows the initial summary; b6e9b5ea-4368-4974-ba22-446f7b58afc5.png shows the saved alert results; e8ffcef6-f63d-4aa8-9000-1b5ec14ec7e4.png shows raw Event 5152 XML.

Final dashboard with Kali scan evidence

The saved SOC Lab - Windows Security Overview dashboard now contains four panels. The global time picker shows Last 7 days. The new Blocked TCP Scan Activity table summarizes the selected historical period, rather than applying the alert threshold.



Figure 9. Final saved dashboard on 20 September 2026 with all four panels.

Panel	Observed result
Failed Logons by Account	52, 31, 9 and 6 events; total 98
Failed Logons Over Time	Timeline retains the earlier test activity
PowerShell Encoded Command Activity	Two process records remain visible
Blocked TCP Scan Activity	405 blocked events; 98 distinct destination ports

The scan row spans 15:55:33.682 to 16:10:25.753 UTC on 20 September and groups source .30 with destination .20. Compared with the first summary of 206 events, the later dashboard contains 199 additional records. It spans both test periods, while the distinct port count remains 98 because repeat observations of the same ports are counted once.

Panel search configuration

The panel uses the scan query on the preceding pages with earliest=-15m latest=now removed, the where unique_dest_ports >= 20 filter removed, and values(scan_dest_port) AS destination_ports omitted for a compact table. Its time range is linked to the global input. The alert retains its separate 15-minute window and threshold.

This completes the positive test path for all three detections and the dashboard. Further validation can include a below-threshold negative test and controlled throttle checks; DC01 ingestion and broader scan coverage remain separate extensions.

References

Product documentation explains the event and configuration semantics. Lab outcome claims are supported by the screenshots and tests described in this report. References are linked in the editable report and PDF.

[1] Splunk inputs configuration reference

[2] Splunk stats command

[3] Microsoft Security Event 4625

[4] Microsoft powershell exe parameters

[5] Microsoft Sysmon documentation

[6] Microsoft Windows PowerShell logging

[7] Splunk Dashboard Studio time range

[8] Splunk timechart command

[9] Splunk alert throttling

[10] Microsoft New NetFirewallRule

[11] Microsoft Security Event 5152

[12] Microsoft Security Event 5157

[13] Nmap options summary

[14] Splunk rex command

Evidence register

Figure	Original screenshot filename
1	97dfae2e-0d8f-436a-bbd2-f1180cca47e8.png
2	67c9aa28-aa3e-4309-9e6e-92f44b0ae44d.png
3	70914a15-b2cc-4eb6-b752-05a43268ebef.png
4	7b02dcd2-423c-4145-b064-93c277ab6cf4.png
5	bce52fa7-cb51-42f5-852a-3f37d9c2c4d7.png
6	20a53cfb-e9dd-4a9c-9450-d8c65050dda2.png
7	fa69fc41-c1bc-4da3-9977-01fc5e457b44.png
8	2f68081a-bbf2-4d07-b593-caf2a44395ea.png
9	04a53afa-bf0f-4d20-a1c2-f1a3dbf11ca9.png

Additional inspected screenshots include 5f1e0ada-bb99-46dc-935f-d36b5d6a2ea3.png for the scheduled PowerShell result, 55dac88a-ebcb-4a80-9f9a-aea43c147744.png for the failed-logon alert settings, and c1186709-1132-4c4f-988a-b5f0a0da6e52.png for the dashboard static time-range problem.

Version 2.0 extends the original Splunk report with the verified Kali scan, Windows network audit configuration, third scheduled alert and four-panel dashboard as of 20 September 2026.